ChatGPT disse:

Agora entendi o que você quer construir. Vamos deixar de remendar o fluxo antigo e organizar o CHECK BOULER 24h como uma **floresta hierárquica de módulos, inspirada no Active Directory**, com orquestração central, herança controlada de regras e execução descentralizada.

E tem um princípio importante do que você explicou: **não vamos inventar uma camada de "células" desconectada do banco**. A estrutura precisa nascer dos domínios, das tabelas, das permissões e dos relacionamentos que já existem no Supabase. A hierarquia organiza esses recursos; não cria uma segunda fonte de verdade.

1. A arquitetura que você está descrevendo



Essa é a estrutura conceitual. A implementação precisa respeitar o banco existente e distinguir herança de configuração, descoberta de módulos e autorização efetiva.

2. Como funciona a hierarquia pai → filho

Cada nó conhece sua própria posição na árvore, seu pai imediato e os filhos que tem autorização para descobrir. A herança deve ser explícita e controlada.

Conceito inspirado no Active Directory	Aplicação no CHECK BOULER 24h
Floresta	Limite superior de confiança e descoberta entre domínios.
Domínio	Contexto organizacional ou de segurança definido por regras reais do sistema.
OU	Agrupamento hierárquico de módulos, recursos ou unidades administrativas.
GPO	Políticas herdáveis: configurações, requisitos e restrições.
Grupos	Conjuntos de permissões associados a funções e contextos.
Dispatcher Master	Coordena descoberta, identidade, políticas e orquestração.
Orquestrador	Resolve a árvore, verifica dependências e encaminha operações.
Executor	Executa somente as operações autorizadas para seu contexto.
Célula operacional	Implementação real de um módulo e de suas funcionalidades.

A regra de descoberta da árvore

Imagine a seguinte sequência:

1. O Dispatcher Master conhece os domínios que estão registrados e pode consultar.
2. Um orquestrador recebe um contexto autorizado e conhece seus filhos permitidos.
3. Cada filho conhece os descendentes que estão registrados e dentro do seu escopo.
4. Se uma nova ramificação for cadastrada, ela não fica automaticamente acessível a todos.
5. A descoberta só continua quando o pai tem autorização para conhecer aquela ramificação e o filho também tem autorização para acessá-la.

Conhecer a existência de um módulo não significa ter permissão para executar suas operações. Essa distinção será fundamental para a segurança.

3. Herança de políticas e credenciais

Precisamos adaptar a analogia do Active Directory ao Supabase:

- As políticas podem ser herdadas do pai, desde que a regra permita herança.
- Um filho pode receber restrições adicionais, mas não deve ampliar privilégios acima do que o pai e as políticas de segurança permitem.

- Uma negação explícita, uma política de isolamento ou uma restrição de tenant não pode ser ignorada por herança.
- Os executores recebem um **contexto de autorização limitado**, não credenciais privilegiadas irrestritas do Dispatcher Master.
- O Supabase continua validando a identidade, as permissões e as políticas RLS. A hierarquia do frontend não substitui essa validação.

Assim teremos herança de regras sem criar um mecanismo que permita a um módulo operacional elevar seus próprios privilégios.

4. Como isso se conecta ao banco atual

O ponto de partida continua sendo a cadeia canônica de identidade:

`auth.users → people → tenant_memberships → role_assignments → roles → role_permissions → permissions`

A partir dela, vamos mapear os domínios e tabelas existentes para a árvore de módulos. Não criaremos uma tabela ou pasta de “células” só porque o nome parece adequado: primeiro precisamos descobrir quais estruturas já representam esses relacionamentos e quais lacunas realmente existem.

Cada módulo operacional terá sua própria estrutura de arquivos:

- Rotas e registro do módulo.
- Dashboard específico.
- Navegação e permissões próprias.
- Páginas, formulários e operações CRUD.
- Hooks, serviços e repositórios.
- Integrações com outros módulos.
- Testes de autorização, herança e isolamento.

O módulo Candidato continua separado do Recrutamento, mas os dois podem se comunicar por seus contratos e dados autorizados. O módulo Suporte continua sendo apenas Suporte — não será um portal obrigatório de entrada.

5. Ordem de construção

Minha recomendação é seguir esta sequência, sem alterar o banco nem modificar arquivos nesta etapa:

1. **Inventariar a floresta atual.** Mapear tabelas, roles, permissions, tenants, rotas, `ModuleRegistry` e `ModuleRouter`.
2. **Definir o nó raiz.** Especificar responsabilidades e limites do Dispatcher Master sem transformá-lo em um superusuário que ignora RLS.

3. **Definir o contrato pai–filho.** Estabelecer descoberta, herança, negação, isolamento de tenant e comunicação entre módulos.
4. **Mapear os módulos reais.** Vincular cada módulo às tabelas, permissões e rotas que já existem.
5. **Implementar um ramo-piloto.** Validar a arquitetura em um módulo antes de replicá-la para toda a floresta.
6. **Testar a segurança.** Confirmar que filhos não descobrem ou executam recursos proibidos, mesmo com acesso direto por URL ou chamada à API.

A decisão central é esta: **a árvore organiza e orquestra; o banco define a identidade e as permissões; cada módulo executa seu trabalho dentro dos limites herdados.**

Esse será o fundamento da nova arquitetura CHECK BOULER 24h, inspirada na hierarquia do Active Directory, mas adaptada à arquitetura real do projeto. O próximo passo correto é o inventário somente leitura, para desenharmos a floresta a partir do que existe — sem reintroduzir a arquitetura antiga nem criar uma estrutura paralela.